

SENTRA — Weekly Development Plan

Prepared by: Rohnit | **Schedule:** 13 to 15 Week Build Schedule | **Context:** Major Project | Final Year (2027 Batch) | **Specialization:** Cybersecurity

How to Use This Document

This plan breaks the 3 to 3.5 month build (from Section 9 of the Sentra Scoping Document) into weekly targets, each with concrete, ordered steps rather than a one-line milestone. Each week's step table includes a blank Status column; use it to mark steps as done, in progress, or blocked as you go, so at any point you can see exactly where the project stands against the plan.

Cross-references like "Section 5A" or "FR-6" point back to the corresponding section or functional requirement in the Sentra Scoping Document, so the two documents are meant to be read side by side, not separately.

If a week runs long, resist the urge to compress the next week to compensate. Use the Week 4 and Weeks 13-15 buffers for that instead; they exist precisely so a slip in one week doesn't cascade into every week after it.

Month 1: Core RAG + Access Control (Weeks 1-4)

This month builds and proves the one piece the entire project's thesis depends on: that retrieval-layer authorization actually restricts what reaches the language model, before anything else is layered on top of it.

Week 1: Document Generation & Environment Confirmation

Goal: Have full synthetic corpus ready and dev environment confirmed working, so Week 2 can start ingestion immediately.

#	Step Description	Status
1	Confirm the backend and frontend skeletons run locally (health-check route, Vite dev server) and the Supabase project/schema exist, per the setup guide.	
2	Finalize the exact document list: pick 8-10 titles per role (Admin, HR, Legal, Employee/public) covering a mix of short memos and longer policies.	
3	Generate each document's content using Gemini, prompting for realistic tone and enough length variation to test chunking later.	

4	Save the generated content in a mix of formats (PDF, .docx, .txt) so the ingestion pipeline is tested against more than one file type.	
5	Tag each file's intended role in its filename or a manifest file (e.g. corpus_manifest.csv) so Week 2's ingestion script can read the role automatically.	
6	Store the finished corpus in docs/corpus/ locally before it moves to OCI Object Storage during ingestion.	

Week 2: Ingestion Pipeline

Goal: A working script that takes any document in the corpus and produces embedded, role-tagged chunks in Supabase.

#	Step Description	Status
1	Write a parsing function per file type (pypdf or pdfplumber for PDF, python-docx for .docx, direct read for .txt) that returns plain text.	
2	Implement chunking using LangChain's RecursiveCharacterTextSplitter, chunk size approximately 500-800 tokens with roughly 100 token overlap.	
3	Write an embedding function that calls the Gemini text-embedding-004 model for each chunk and returns the vector.	
4	Insert one row per source document into the documents table, and one row per chunk (with its embedding and inherited role tag) into the chunks table.	
5	Run the full pipeline on a single document first, confirm both tables populate correctly and the embedding dimension matches the pgvector column.	
6	Batch-run the pipeline across the full corpus from the manifest file, then spot-check a few chunks in the Supabase table editor for correctness.	

Week 3: Auth, Row-Level Security, and the Retrieval Endpoint

Goal: A user can authenticate, and a retrieval query correctly returns only the chunks their role permits.

#	Step Description	Status
1	Enable the email/password provider in Supabase Auth and disable any others.	
2		

	Manually create the four test accounts (Admin, HR, Legal, Employee) and set each one's role in user metadata.	
3	Write the Postgres Row-Level Security policy on the chunks table so a query only returns rows whose role matches the authenticated user's role (or is public).	
4	Build the FastAPI /query endpoint: verify the incoming JWT, embed the user's question via Gemini, and run the pgvector similarity search under RLS.	
5	Test the endpoint via curl or Postman, logged in as each of the four roles, and confirm the returned chunks match the access matrix in Section 5A of the scoping document.	
6	Stub the Model A/B/C config flag in the backend now, even if only Model C is wired up, so Month 2's switch work has a place to plug into.	

Week 4: Full Role Testing + Buffer

Goal: Confidence that retrieval-layer authorization is airtight before building anything on top of it.

#	Step Description	Status
1	Build a small test matrix: for each of the 4 roles, run 3-4 sample queries and record which chunks came back.	
2	Compare every result against the access matrix in Section 5A. Fix any mismatch immediately, this is the one piece nothing else in the project should be built on top of until it's solid.	
3	Confirm RLS also correctly handles the "no matching content" vs "access denied" distinction from FR-15, even at the raw endpoint level, before it becomes a frontend problem in Week 6.	
4	Use any days left this week as buffer. If ingestion or RLS from Weeks 2-3 slipped, absorb it here, don't start Month 2 with a shaky Month 1.	
5	Write a short note capturing what you learned this month, this becomes raw material for the project report in Week 12.	

Month 2: Web App + Cloud Deployment (Weeks 5-8)

This month turns the working backend into a usable, deployed application, and introduces the Model A/B comparison configurations alongside the default Model C.

Week 5: Generation Endpoint

Goal: A full query-to-answer round trip, API-only, no UI yet.

#	Step Description	Status
1	Write the prompt template that combines the retrieved chunks with the user's question for the Gemini generation call.	
2	Extend the /query endpoint so it calls generation immediately after retrieval and returns the final answer.	
3	Handle the explicit access-denied case (FR-6): if RLS returns zero chunks purely because of role restriction, return a clear "Access Denied" message rather than letting the LLM improvise from an empty context.	
4	Handle the separate "no relevant documents found" case (FR-15): distinguish a genuinely empty result from a role-restricted one, since these need different audit log outcomes later.	
5	Test the full round trip via curl/Postman across all four roles, and manually try a couple of Category H ("fabrication under denial") style prompts to see how the raw endpoint currently responds.	

Week 6: Frontend Shell

Goal: A working login and chat UI hitting your real backend.

#	Step Description	Status
1	Build the login page against Supabase Auth, and store the returned JWT in React state (Context or a simple provider).	
2	Attach the JWT as an Authorization header on every API call.	
3	Build a minimal chat interface: message list, input box, and a call to the /query endpoint on submit.	
4	Add a small role indicator in the UI so it's always visible which account you're testing as, useful for your own sanity and later for the viva demo.	
5	Test the full flow in the browser, logged in as each of the four roles.	

Week 7: Dashboards + Document Upload

Goal: Admin can upload a new document through the UI, and it becomes queryable.

#	Step Description	Status
1	Build a simple per-role dashboard view (at minimum: who's logged in, their role, maybe a query history).	
2	Build the upload UI (Admin-only), with fields for department/role tag and sensitivity level.	
3	Wire the upload form to a backend endpoint that stores the raw file in OCI Object Storage and then calls the Week 2 ingestion pipeline (refactor it into a reusable function if it was a standalone script).	
4	Implement FR-14 (deletion): an endpoint that removes both the OCI file and its associated Supabase rows, so nothing orphaned remains queryable.	
5	Test end to end: upload a brand-new document, then immediately query it as the appropriate role to confirm it's retrievable.	

Week 8: Deployment

Goal: The whole system live on Render and Vercel, running in all three authorization configurations.

#	Step Description	Status
1	Connect the backend repo to Render, set the environment variables in Render's dashboard, and confirm the deployed health-check route responds.	
2	Connect the frontend repo to Vercel, set its environment variables, and confirm the deployed app loads and can reach the backend.	
3	Implement the actual Model A/B/C switch as a backend config value: Model A skips authorization entirely, Model B checks role in the FastAPI layer after retrieval, Model C uses the RLS path already built.	
4	Run one full smoke test in production: log in as each role, run a query, try an upload, and switch between all three models to confirm each behaves as designed.	

Month 3: Evaluation, Hardening & Delivery (Weeks 9-12)

This month turns the deployed system into a defensible research result: the attack suite, the evaluation harness, the results table, and the written answers to the five research questions.

Week 9: Audit Logging + Attack Suite Authoring

Goal: Every query is logged, and the fixed attack query set exists as a reusable, structured file.

#	Step Description	Status
1	Create the access_log table in Supabase (user, role, query text, outcome, retrieved chunk IDs, timestamp).	
2	Log every query, allowed, denied, and no-content, so FR-7 and FR-10 are both satisfied.	
3	Write out the 20-30 attack queries across all 8 categories from Section 5C, including the fabrication-under-denial category.	
4	Store the attack set as a structured file (JSON or CSV) with a category label per query, so the evaluation script in Week 10 can loop over it programmatically.	

Week 10: Evaluation Harness

Goal: A script that runs the full attack set against all three models and produces raw metrics.

#	Step Description	Status
1	Write a Python script that loops over every attack query, running it three times, once per model (A, B, C).	
2	For each run, capture the retrieved chunk IDs with their true role labels, the final response text, and the response latency.	
3	Compute the six core metrics per model (URR, Retrieval Leakage, Generation Leakage, ASR, Retrieval Precision, Latency Overhead) plus the separate fabrication flag from Category H.	
4	Save all raw results to a CSV or JSON file, this is the source data for next week's analysis and for the report's appendix.	

Week 11: Analysis + Hardening

Goal: A finished results table, written answers to the five research questions, and any bugs the attack suite exposed are patched.

#	Step Description	Status
1	Build the comparison table (Models A/B/C across all six metrics) from the raw data.	
2	Draft written answers to RQ1 through RQ5, grounded directly in the numbers, not just the expected pattern from Section 5D.	
3	If Model C underperformed anywhere, treat it as a real finding: patch the underlying bug, re-run just the affected queries, and note the fix in the report rather than silently re-running until it looks clean.	
4	Do a final manual security pass: confirm .env was never committed, confirm RLS policies exist on every sensitive table, confirm JWTs expire and are checked on every request.	

Week 12: Report + Viva Prep

Goal: A complete, submittable project report and a rehearsed demo.

#	Step Description	Status
1	Write the project report using the scoping document as the source of truth for architecture, decisions, and rationale, and the Week 11 results table for the evaluation section.	
2	Prepare a viva walkthrough: a short architecture explanation, a live demo across all four roles, a look at the audit log, and a summary of the evaluation results.	
3	Rehearse the demo end to end at least once, including deliberately triggering an access-denied case live, since that best demonstrates the whole thesis of the project.	

Weeks 13–15 (Buffer): Refinement Only

Goal: Absorb any slippage without adding scope.

#	Step Description	Status
1	If every week above landed on time, use this period to tighten the report's writing and polish the demo, not to add new features.	
2	If something slipped, this is where it gets caught up, in the order it was originally due, not in order of what's most interesting to work on.	

Companion document to the Sentra Scoping Document. Use both together: this plan for weekly execution and tracking, the scoping document for architecture, requirements, and evaluation design.